

WIA1002/WIB1002 DATA STRUCTURE



SEM 2 2025/2026

OCC 11

LECTURER DR FAIRUZ AMALINA BINTI NARUDIN

NAME	MATRIC NUMBER
NADIYAH AQILAH PUTRI	24211204
YUDA SULTHAN KAMAL	24212069
MUHAMMAD AULIA ZULFADHLI SJAMSUDIN	24222534
MUHAMMAD FAEYZA ASHIRA	24226508
ABDUL MALIK RASYID	24231329

TABLE OF CONTENTS

TABLE OF CONTENTS	2
PROBLEM STATEMENT	3
1.1 Introduction.....	3
1.2 The Problem Statement.....	3
1.3 Project Objective.....	4
APPROACH	4
2.1 Architectural Design and Information Hiding.....	4
2.2 Catalogue Indexing and Search Strategy.....	4
2.3 Transaction Processing Logic.....	4
2.4 Defensive Programming and Interface Resilience.....	5
DATA STRUCTURES USED	5
3.1 Binary Search Tree (BST).....	5
3.2 Stack.....	5
3.3 Why These Structures?.....	5
TESTING	6
4.1 Testing Strategy Overview.....	6
4.2 Module-by-Module Test Log.....	6
4.3 Testing Conclusion.....	8
CONCLUSION	8

PROBLEM STATEMENT

1.1 Introduction

In an academic library setting, managing data effectively involves balancing two distinct operations: handling a large, growing catalogue of resources and maintaining clear records of user activities. A university library system relies heavily on fast cataloguing and immediate retrieval of book information. When a student borrows a book, the system must also capture this transaction immediately.

Conventional linear data structures (such as standard arrays or linked lists) become highly inefficient as data scales. When thousands of academic titles and user transactions are processed simultaneously, systems utilizing basic search and storage mechanisms suffer from extreme performance degradation. To provide a viable solution for students and administrators, a modern library system must shift toward optimized data structures tailored to specific data access patterns.

1.2 The Problem Statement

To build an efficient and scalable library management system, several critical technical bottlenecks must be addressed:

- **Inefficient Search Operations:** Searching through a massive, flat catalogue sequentially results in an $O(n)$ time complexity, which is computationally unsustainable. To ensure instant access to records, the system requires a hierarchical model that indexes book records by their unique ISBN numbers to achieve a logarithmic efficiency of $O(\log n)$.
- **Disorganized Activity Tracking:** When students interact with the library, they expect to review their most recent actions first. Processing transactions chronologically via standard linear structures makes displaying the latest activity difficult. The system needs an explicit Last-In-First-Out (LIFO) access pattern to manage borrowing histories dynamically.
- **Lack of Information Hiding:** Software systems frequently become fragile when the user interface directly modifies or interacts with raw backend data structures. Exposing internal tree configurations or stack frames directly to the terminal layer compromises data integrity and violates foundational object-oriented programming guidelines.
- **Fragility Against Invalid User Input:** Simple terminal applications are highly vulnerable to crashing when subjected to human error during data entry. The system lacks resilience unless it can gracefully handle operational anomalies, such as non-integer ISBN strings, blank input fields, or requests for empty search results.

1.3 Project Objective

The core objective of this project is to engineer a functional, robust, and menu-driven Smart Library System in Java that resolves these architectural challenges.

APPROACH

2.1 Architectural Design and Information Hiding

To resolve the vulnerability of exposing internal data structures to the user interface, the system adopts a modular, interface-driven architecture. The core operations are strictly abstracted through the LibraryADT interface. This design enforces absolute information hiding by ensuring the main operational class (SmartLibrary) interacts solely with high-level method signatures. Consequently, the internal memory management of tree nodes and stack frames remains completely encapsulated and protected from unauthorized external modifications.

2.2 Catalogue Indexing and Search Strategy

To overcome the severe computational bottleneck of sequential searching, the catalogue is engineered using a recursive data indexing strategy. Every newly added book entity is dynamically routed to a specific memory pointer based on the numerical value of its ISBN. For data retrieval, the system isolates the algorithmic logic into a dedicated RecordFinder module. This approach allows the system to systematically divide the search space in half during every traversal step, successfully achieving the required logarithmic efficiency for instant record access.

2.3 Transaction Processing Logic

To rectify the issue of disorganized activity tracking, the system enforces a strict Last-In-First-Out (LIFO) workflow for handling borrowing transactions. Rather than appending borrowing records sequentially to a standard list, the operational logic pushes each borrowed book reference directly to the top of an isolated memory stack. When a user queries their history, the system executes a reverse iteration algorithm. This approach natively guarantees that the most recent transactions are displayed first, entirely eliminating the need for secondary sorting operations.

2.4 Defensive Programming and Interface Resilience

To protect the terminal application from catastrophic failures caused by unpredictable human data entry, defensive programming paradigms are implemented directly at the user interface layer. The input processing system is fortified with continuous validation loops and strict exception-handling mechanisms. This strategy ensures that operational anomalies such as non-integer ISBN inputs, blank fields, or missing catalog records are gracefully intercepted. Instead of crashing the program, the system safely discards the invalid buffer and reprompts the user, ensuring uninterrupted runtime stability.

DATA STRUCTURES USED

3.1 Binary Search Tree (BST)

The catalogue is stored as a BST where each node is a Book object. The BST property is maintained on ISBN: every node in a left subtree has a smaller ISBN than its parent, and every node in a right subtree has a larger ISBN.

- Insert: $O(\log n)$ average, $O(n)$ worst case on a skewed tree.
- Search: $O(\log n)$ average; the recursive implementation in RecordFinder halves the search space at every node.
- In-order traversal: produces books sorted by ISBN, used by the View Catalogue feature.

3.2 Stack

Borrowing history is maintained as a `java.util.Stack<Book>`. The LIFO (Last-In-First-Out) property means the most recently borrowed book is always on top, which directly matches the requirement that history be displayed most-recent-first.

- `push(book)`: called when a student borrows a book; $O(1)$.
- `peek()`: returns the top element without removing it; $O(1)$.
- `pop()`: removes and returns the top element for the return-book feature; $O(1)$.
- `show()`: iterates from top to bottom to display LIFO order.

3.3 Why These Structures?

The reason for the use of these structures can be explained by the table below:

Requirement	Structure	Reason
Fast lookup by ISBN	BST	$O(\log n)$ vs $O(n)$ for arrays/list
Most recent book first	Stack	LIFO is the natural fit
Hide internal structures	Interface (ADT)	Enforces information hiding

TESTING

4.1 Testing Strategy Overview

The objective of this testing phase is to validate the correctness, efficiency, and stability of the Smart Library System. Testing is split into two primary methodologies:

- *Unit Testing*: Isolating individual components (BookBST, BorrowStack, and RecordFinder) to verify independent algorithmic logic.
- *Integration & Interface Testing*: Verifying that the data flows safely through the LibraryADT interface into the SmartLibrary runtime engine without crashing or leaking internal structures.
-

4.2 Module-by-Module Test Log

A. Catalogue Management (Book.java & BookBST.java)

Test ID	Component	Test Objective	Input Steps	Expected Output	Status
TC-BST-01	BookBST	Verify root node creation.	Insert first book: ISBN 50.	The node is designated as root. Left and right pointers are null.	PASS
TC-BST-02	BookBST	Verify left subtree insertion (Smaller ISBN).	Insert second book: ISBN 30 (when root is 50).	The algorithm traverses left and places ISBN 30 as root.left.	PASS
TC-BST-03	BookBST	Verify right subtree insertion (Larger ISBN).	Insert third book: ISBN 70 (when root is 50).	The algorithm traverses right and places ISBN 70 as root.right.	PASS

B. Borrowing History (BorrowStack.java)

Test ID	Component	Test Objective	Input Steps	Expected Output	Status
TC-STK-01	BorrowStack	Handle empty history display state.	Call show() before any books are borrowed.	Prints: "Borrowing history is empty." cleanly without errors.	PASS
TC-STK-02	BorrowStack	Verify LIFO display order sequence.	Push Book A (ISBN 10), then Push Book B (ISBN 20). Call show().	Book B (most recent) displays at index [1] on top; Book A displays below it.	PASS
TC-STK-03	BorrowStack	Verify stack pop/undo function.	Call pop() after pushing Book A and Book B.	Removes Book B from the stack and returns its reference.	PASS

C. Search Engine (RecordFinder.java)

Test ID	Component	Test Objective	Input Steps	Expected Output	Status
TC-FND-01	RecordFinder	Locate an existing item recursively.	Search for ISBN 30 (inserted previously in left subtree).	Traverses root.left, successfully returns the Book metadata.	PASS
TC-FND-02	RecordFinder	Graceful handling of empty search results.	Search for an ISBN not in the tree (e.g., 999).	Reaches a base case of null, returning an explicit "not found" message instead of breaking.	PASS

D. System Integration & Interface UI (LibraryADT.java, SmartLibrary.java, Main.java)

Test ID	Component	Test Objective	Input Steps	Expected Output	Status
TC-INT-01	LibraryADT	Confirm Information Hiding compliance.	Attempt to write lib.catalogue.root directly from Main.java.	Compilation error occurs, proving internal data trees are private.	PASS

TC-INT-02	SmartLibrary	Handle out-of-bounds numerical menu options.	Input option choice 6 or -1 at the console prompt.	Displays "Invalid option." and reprompts the user without crashing.	PASS
TC-INT-03	SmartLibrary	Verify programmatic termination.	Input option choice 5 at the console prompt.	Breaks the while loop, closes the Scanner, and terminates execution cleanly.	PASS

4.3 Testing Conclusion

All integrated code modules compile and pass execution test constraints. The Binary Search Tree behaves optimally with logarithmic traversal boundaries, the transaction history maintains exact chronological LIFO structures, and the concrete implementation prevents runtime faults through custom terminal feedback.

CONCLUSION

This project successfully demonstrates the design and implementation of a Smart Library System in Java, engineered to address the core inefficiencies inherent in conventional linear data structure approaches. By leveraging a Binary Search Tree for catalogue indexing and a Stack for borrowing history management, the system achieves logarithmic search efficiency and a natural Last-In-First-Out transaction display, both of which are critical requirements for a scalable academic library environment.

The adoption of an interface-driven architecture through the LibraryADT abstraction layer ensures strict information hiding, protecting internal data structures from unauthorized external access and upholding sound object-oriented design principles. Furthermore, the integration of defensive programming techniques at the input layer guarantees runtime stability against unpredictable user behaviour, preventing system crashes due to invalid or malformed entries.

All defined test cases across the BST, Stack, search engine, and system integration modules returned a PASS status, confirming that the system behaves correctly under both normal and edge-case conditions. The modular design also ensures that individual components can be

extended or replaced independently, making the system adaptable for future enhancements such as AVL tree balancing for worst-case performance guarantees or persistent database integration.

In conclusion, the Smart Library System fulfills all project objectives — delivering a functional, robust, and menu-driven application that applies appropriate data structures to real-world library management challenges.